Joshua Ko

925-519-9676 | joshuadohako@gmail.com | linkedin.com/in/joshua-doha-ko/ | github.com/kojosh

Education

University of California, Irvine — Irvine, CA
B.S., Computer Engineering — September 2024 – March 2027
GPA: 3.86, Dean's Honor List (5 Quarters)

- Relevant Coursework: Data Structures & Algorithms, Software Engineering in C, Network Analysis, Electronics, Organization of Digital Computers, Digital Systems, Computer Networks, System Software, Machine Learning

Skills

Languages: C++, C, Java, Python, Verilog, VHDL, MIPS Assembly
Frameworks/Libraries: Flask, YOLOv8, OpenCV, LeRobot
Tools/Platforms: Adobe Photoshop, Git, GitHub, Linux, Conda, Hugging Face Hub, Raspberry Pi, Arduino, ESP32, Ollama
Awards: Gold Key Award (Scholastic Art & Writing Competition 2020)

Experience

SDE Intern — September 2026 – Present
Amazon Web Services — *Seattle, WA*

- AWS, EC2

Software Intern — July 2026 – August 2026
CAC Pension — *San Ramon, CA*

- Built a full stack CLI app using a local LLM via Ollama and Qwen 3.6 for automated PDF form filling and drafting emails using PyMuPDF and Microsoft Graph.
- Designed an asynchronous job queue system in Python and SQLite to process PDF form-filling requests from multiple employee workstations, using a single-consumer worker pattern.
- Implemented an offline data pipeline to secure clients' data, engineering a Sharepoint ingestion/export workflow to ensure zero data loss with no network dependency.

Undergraduate Researcher — February 2026 – July 2026
Advanced Integrated Cyber-Physical Systems (AICPS) Lab — *UC Irvine*

- Developed Python automation scripts, configured Conda environments and LeRobot pipelines for deployment, testing, and benchmarking of machine learning control policies
- Build and manage multimodal robotics data pipelines, curating 2 datasets of 100 teleoperated episodes total (80k frames at 30 FPS) with synchronized 6-DoF joint trajectories and RGB video for ML model training
- Fine-tune Vision-Language-Action (VLA) models using the Physical Intelligence π_0, $\pi_{0.5}$, and NVIDIA GR00T N1.5 architectures and implement evaluation workflows for embodied AI inference and performance benchmarking
- Design projected-gradient-descent (PGD) adversarial attacks on Vision-Language-Action (VLA) robotic policies by injecting bounded universal perturbations into proprioceptive sensor inputs, degrading manipulation task performance while remaining within normal calibration variance

Projects

Campion
Diamond Hacks 2026 Best UI/UX Hack

- Collaborated with a team to build a full-stack web application using React, FastAPI, and Mapbox GL to enable real-time dispersed campsite discovery with interactive geospatial visualization and routing
- Integrated multi-agent browser automation (Browser Use SDK) to asynchronously scrape, aggregate, and validate data from topography, land-use regulations, and community sources
- Developed RESTful APIs and backend services with FastAPI (Python), implementing parallel processing, API integration, and environment-based configuration for scalable data workflows
- Implemented responsive frontend architecture with React, Vite, and Tailwind CSS, winning the Best UI/UX Hack out of 366 participants

TARS Autonomous Robot
UCI Zotbotics Level 3

- Contribute to the electrical systems team in the design, assembly, and debugging of an autonomous mobile robot
- Develop electrical schematics in KiCad and designed a centralized power distribution system to stepper motors, drivers, and microcontrollers
- Develop low-level modular embedded software in C++ (Arduino) for stepper motor control, implementing driver interfaces and motion control logic
- Collaborated with cross-functional teams to implement stepper motor control using PWM and USB serial communication interfaces

Pipelined MIPS Processor
Intro to Digital Logic Lab Project

- Design and verify hardware modules for a pipelined MIPS processor using Verilog, including instruction memory, register file, ALU, and data memory
- Develop and execute testbenches to validate functional correctness and performance of each pipeline stage
- Debug and simulate designs using Xilinx Vivado, resolving timing and logic issues
- Implement instruction pipelining to achieve approximately 35–45% performance improvement in instruction throughput compared to a single-cycle baseline on FPGA